What a Loop Buys: Parameter Efficiency on Text, Extendable Depth on a Serial Task

Qirun Li *qli858246@gmail.com*
Independent researcher

Abstract

When does running a block of layers again work as test-time compute? A looped transformer adds depth without adding parameters, and two benefits are claimed for it: that it behaves like a larger model, and that more loops at inference solve harder problems without a written chain of thought. We test both in a pre-registered study. On web text we trained a grid of 111 models from 10M to 80M parameters with dense references up to 160M, varying where the loop sits, how often it runs and how it is backpropagated, and extended it in model size, data and learning rate. On a lookup-chain task that needs serial steps we compared looped models with dense models of equal parameters and of equal executed depth, with a written chain of thought and with pause tokens. None of the four pre-registered hypotheses held as stated, and the pre-registered effective-parameter exponent is not identified. In an exploratory analysis without a fitted law, a parameter of a looped middle block is worth 0.6–1.0 dense parameters at 10–20M with up to 20 training tokens per parameter and about 1.3–1.5 at 40M–160M with 20–40, rising with the tokens per parameter; at equal training compute and a shared learning rate the dense model wins every measured comparison, by a margin that shrinks with data. On text the looped block runs a fixed-length program, a cycle or a fixed point, depending on training, and gains nothing from extra loops. On the chain task the same architecture computes the chain step by step inside the loop and solves longer chains when run longer (a model trained with 8 loops on chains of at most 8 steps goes from 0.04 to 0.83 accuracy on 12-step chains when run with 16 loops), and for most training regimes its own confidence tells it when to stop. Dense models of equal depth do not extend beyond the training range, and neither does a chain of thought on this task, whose prompt grows with the chain. At deployment and equal loss, a middle block looped 4 times saves at most 19% of the weight memory and costs about 2.2–2.5 times the KV cache and the decoding time. Code, configurations, training logs and results are released.

1 Introduction

A looped, or recurrent-depth, transformer applies one block of layers several times in a row (Dehghani et al., 2019; Geiping et al., 2025; Zhu et al., 2025). Running a block r times costs r times its compute and adds no parameters. Two benefits are expected in return. The first is *parameter efficiency*: a looped model should behave like a larger dense model, which matters wherever memory or bandwidth rather than arithmetic limits deployment. The second is *latent reasoning*: if the loop iterates on a problem, running more loops at inference should solve harder problems without writing out a chain of thought, a form of test-time compute that lives inside the network.

Both claims are hard to check from the literature. Studies differ in data, tokenizer, training recipe and model family; seeds are rarely reported; and the quantity of interest, how much one more loop is worth, is usually read off a single fitted scaling law (Schwethelm et al., 2026; Prairie et al., 2026; Chen et al., 2026a). Evidence for test-time scaling comes from two directions that are seldom put side by side: large models trained with a random number of loops, which improve on reasoning benchmarks as more loops are run at inference, up to a task-dependent saturation (Geiping et al., 2025), and small models on synthetic tasks, which solve longer problems when run longer (Fan et al., 2025; Kohli et al., 2026; Zhang et al., 2026).

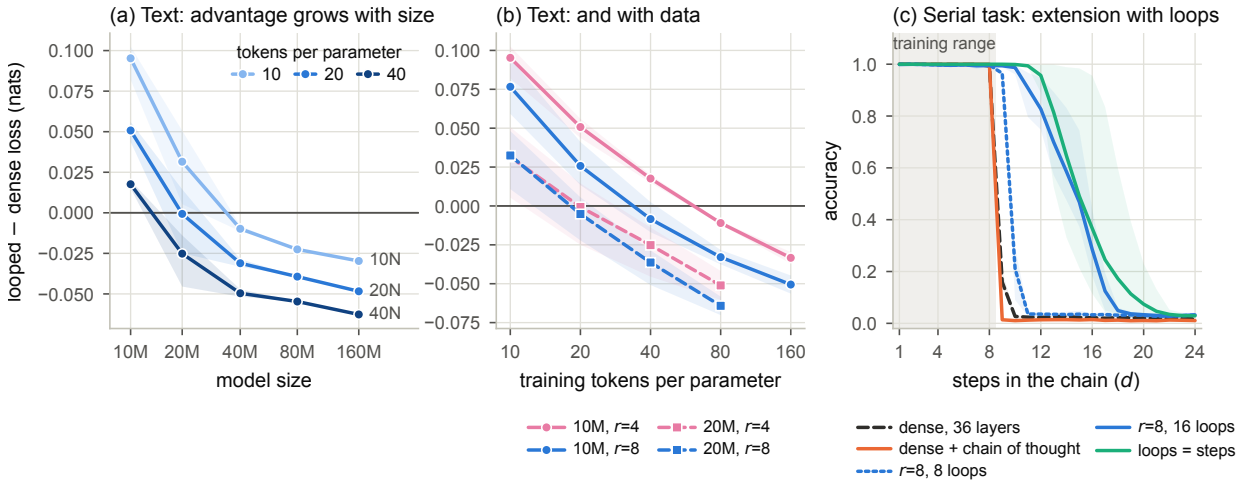


Figure 1: What a loop buys, in one figure. **(a)** On web text, at equal parameters and training tokens, a middle block looped 4 times goes from worse than the dense model at 10M to better at every budget from 40M on (looped minus dense validation loss; budgets of 10, 20 and 40 training tokens per parameter; band: range over seeds). **(b)** The advantage also grows with the training tokens per parameter, here at 10M and 20M for middle blocks looped 4 and 8 times. **(c)** On chains of table lookups, trained on 1–8 steps (shaded), a dense model with the executed depth of 8 loops (36 layers) stops at the training range, and so does a dense 8-layer model that writes the chain out, whose prompt grows with the chain; a looped model trained with 8 loops and run with 16 solves longer chains, and so does one trained with as many loops as steps (mean over five seeds; band: range).

We measured both claims in one study, with one architecture family and one recipe. The hypotheses and the analysis of the main grid were written down before its first training run, and every later plan was frozen before its own runs. On web text, a grid of 111 runs varies model size (10M to 80M looped, dense references to 160M), where the loop sits (a middle block or the whole stack), how often it runs ($r = 2, 4, 8$) and how it is backpropagated (through all loops or through the last half), with three seeds at the small sizes and cooldown branches at three data budgets. Extensions add a 160M looped model, up to 160 training tokens per parameter, random loop counts, and nine supplementary experiments whose plans were frozen before they ran. When the text results left the second claim untested, we built a task that needs serial steps, a chain of lookups in fixed random tables, froze its design and predictions, and ran it the same way: looped models against dense models of equal parameters and of equal executed depth, against a dense model that writes the chain out, and against pause tokens.

The two settings give different answers (Figure 1). On text, the loop buys parameter and data efficiency, not compute efficiency: measured without a fitted law, a parameter of the looped block is worth at most about one dense parameter at 10–20M with up to 20 training tokens per parameter and about 1.3–1.5 dense parameters at 40M–160M with 20–40, the value grows with the tokens per parameter, and at equal training compute and a shared learning rate the dense model wins every measured comparison, by a margin that shrinks with data. Depending on how it is trained, the looped block runs a fixed-length program, a cycle or a fixed point, and none of these gains from extra loops at inference. On the chain task, the same architecture computes the chain step by step inside the loop, solves chains longer than it was trained on when run longer, and for most training regimes can tell from its own confidence when to stop. A written chain of thought is perfect within the training range at a third of the looped model's compute and does not extend at all; on this task, whose prompt grows with the chain, that is a failure to generalise to longer inputs.

We read the difference as a matter of three conditions, which the chain task meets once the model is trained for them: *headroom*, meaning that errors come from too few serial steps rather than from missing knowledge; *extendable iteration*, meaning that the loop can run longer without destroying its state; and a *stopping signal*.

On web text at these sizes we saw no sign of the first. This reading is an interpretation of two settings, not a tested law, and Section 8 discusses what would test it.

Our contributions are:

- **A pre-registered measurement of what a loop is worth on text** (Section 4). None of the four hypotheses held as stated. The pre-registered exponent φ of the effective-parameter law $N_{\text{eff}} = N_{\text{once}} + r^{\varphi} N_{\text{rec}}$ is not identified, because the dense law it is read from is weakly identified. Measured without that law, the value of a loop depends on model size and on training tokens per parameter, an interaction that effective-parameter laws whose loop gain does not depend on the training tokens do not capture. At equal training compute the dense model wins all 258 measured comparisons, up to 640 tokens per parameter, and the two reach parity only at the smallest size and the most data, with the looped model's own learning rate.
- **What the looped block computes on text** (Section 5). Depending on training, it runs a fixed-length program, a periodic cycle or a contraction to a fixed point; none gains from extra loops.
- **Loops as extendable depth on a serial task** (Section 6). With dense controls of equal executed depth and probes of every intermediate value, looped models compute the chain step by step (one step per loop when the model has a loop for every step) and solve longer chains with more loops at inference. The learning rate decides whether this is learned at all. In a controlled comparison of answer-free stopping rules, the model's own confidence is the best rule that actually stops, and a training regime with a few random extra loops gives a model that both extends and stops.
- **A comparison with reasoning written out** (Section 6.6). On this task, a dense model that writes the chain out as tokens is perfect within the training range and fails beyond it from the first written token on, a failure to generalise to longer prompts that leaves open what happens when the input length is fixed. Pause tokens add nothing.
- **The cost at deployment** (Section 7), and an open record: the pre-registration, a dated log of every deviation and correction, and the results and training logs of all 232 language-model runs and 135 synthetic-task runs (Section 10).

2 Related work

How much a loop is worth. Sharing parameters across layers is an old way to trade compute for parameters (Dehghani et al., 2019; Lan et al., 2019; Bae et al., 2025a). Kaplan et al. (2020, App. D.2, Fig. 17) found that transformers which re-use parameters "perform slightly better when comparing models with equal parameter count, but slightly worse when accounting for reuse and comparing per FLOP"; our results agree in both directions and add how the balance moves with model size and data. The effective-parameter form we test, $N_{\text{eff}} = N_{\text{once}} + r^{\varphi} N_{\text{rec}}$, and the layout with two unshared layers before and after the loop come from Schwethelm et al. (2026), who estimate $\varphi = 0.46$ (95% interval 0.41–0.53) on FineWeb-Edu with up to eight loops; our pre-registration was built to be comparable with theirs. Parcae (Prairie et al., 2026) finds that looping lowers perplexity at matched parameters and tokens, and that at a fixed FLOP budget looping and data should be increased in tandem; Lee et al. (2026) that dense looped models scale worse than non-looped ones at equal FLOPs; Chen et al. (2026b) that tying the weights of a deeper model costs a roughly constant factor of compute in single-epoch training, while with repeated data the best loop count grows with compute; for dense looped models and looped mixtures of experts alike, a bounded form extrapolates better than r^{φ} to 16 loops (Chen et al., 2026a), and in single-seed ablations of a compute-matched looped mixture of experts, looping the middle half of the layers beat looping all of them by less than 0.01 nats (Wang et al., 2026b). In a controlled appendix of Ouro (Zhu et al., 2025) a dense model of the same depth beats the looped one in every comparison, and plain recursion is worse than a vanilla model in validation loss at matched compute in Bae et al. (2025b). A 1.4B looped model trained on 310B tokens (210B of pretraining, about 13 passes over a 16B-token corpus, then 100B of mid-training) beats a dense model with the same parameters and data on all 12 of its benchmarks at about four times the training compute per token, while a 3.9B dense model that matches its inference compute at three loops is

ahead or level on every benchmark (Marchenko et al., 2026). When data is limited, loops may pay most: recursive transformers beat standard ones on downstream evaluations when pretraining on 10M–100M words (Gülbahar et al., 2026). None of these studies varies model size and tokens per parameter separately with several seeds, which is what our grid adds; Section 4.2 restates their equal-parameter results in a common unit.

What the loop computes. Huginn (Geiping et al., 2025), trained with a random loop count, has latent trajectories that, in qualitative examples, settle into fixed points or orbits or drift steadily in one direction; Logan (2026) reports per-token convergence to a fixed point in a small depth-recurrent model; deep equilibrium models make the fixed point the definition of the output (Bai et al., 2019), and those whose fixed point does not depend on the starting state make better use of extra test-time computation (Anil et al., 2022). Saunshi et al. (2025) find that the gains of looped models concentrate on reasoning tasks. We read the state after every loop with the model's own output layers and with learned maps in front of them, as the logit lens (nostalgebraist, 2020) and the tuned lens (Belrose et al., 2023) do for the layers of a dense model, for three training regimes.

Serial reasoning and extrapolation. Recurrent networks solve harder instances than they were trained on by running more iterations, and can "overthink" (Schwarzschild et al., 2021; Bansal et al., 2022). Looped transformers with an input-dependent number of loops generalise to longer inputs on algorithmic tasks (Fan et al., 2025), and on top of position embeddings made for the task, recurrent layers with input injection help arithmetic generalise further to longer numbers (McLeish et al., 2024). Recurrent-depth transformers extrapolate to deeper chains of composed facts with more loops at test time (trained on up to 12 hops, they reach 14–19), while vanilla transformers of equal parameters or equal FLOPs do not (Kohli et al., 2026); and the positions a loop resolves per iteration scale with the training length divided by the training loop count, within a task-dependent range (Zhang et al., 2026). Shallow transformers can also learn shortcuts for serial computations whose steps compose associatively (Liu et al., 2022). Our synthetic results agree with these studies; we add several seeds, probes of every intermediate value after every loop, the effect of the learning rate on whether the task is learned at all, a controlled comparison of stopping rules, and a comparison, on the same problems, with a chain of thought (Wei et al., 2022), here learned from scratch as in scratchpad training (Nye et al., 2021), which adds serial computation to a fixed-depth transformer (Merrill & Sabharwal, 2024), and with pause tokens (Goyal et al., 2024), which give it extra parallel computation; filler tokens of this kind help on some algorithmic tasks (Pfau et al., 2024). RELAY (Yu et al., 2025) aligns loop iterations with chain-of-thought steps and uses a looped model's length generalisation to produce chains for longer problems, and continuous thoughts (Hao et al., 2025) feed the hidden state back as input. Transformers that write tokens are known to generalise poorly to inputs longer than those seen in training (Zhou et al., 2024; Cho et al., 2024), which is how our chain-of-thought baseline fails.

When to stop. Adaptive computation time (Graves, 2016), used by Universal Transformers, and PonderNet (Banino et al., 2021) learn a halting distribution during training; Mixture-of-Recursions learns a recursion depth per token (Bae et al., 2025b). Fan et al. (2025) report results with the ground-truth number of steps and propose, as an alternative, the loop count up to a maximum at which the model is most confident in its own output, chosen per test set or per problem; stopping when a confidence threshold is first met is mentioned there as a variant. We compare answer-free stopping rules applied after training (thresholds on the state change, on the prediction and on the confidence, and votes over loop counts) on the same checkpoints across training regimes, with the rules and thresholds fixed in advance and selected on a separate problem set.

Cost at inference. Without sharing, a looped model's KV cache and decoding time grow with the loop count: two loops take about twice the KV memory and 1.96 times the latency in Wu et al. (2025), and plain looped decoding steps take 2.3, 3.4 and 5.7 times as long at 2, 3 and 5 loops in Monea et al. (2026). Ouro (Zhu et al., 2025) shares the cache across loops during decoding only, since processing the prompt needs a cache per loop; Huang et al. (2026) share it and find that full backpropagation through a fixed depth prevents sharing; and Monea et al. (2026) pretrain looped models in which only the first loop writes a cache

Table 1: Model sizes and training settings on text. Size names give the approximate non-embedding parameters; N also counts the output head (Porian et al., 2024), which is large at small widths with a 16k vocabulary. The looped models of a size have the same unique layers plus a $2w^2$ injection map (under 2% of N). The learning rate is that of the embedding and output layers, frozen after a 33-run sweep; the hidden layers use 1.22 times it. Warm-up is $2N$ tokens.

Size	Width w	Heads	N	Tokens per step	Learning rate	Seeds (dense / looped)
10M	320	5	15.4M	81,920	2.19×10^{-3}	3 / 3
20M	448	7	26.8M	131,072	1.53×10^{-3}	3 / 3
40M	640	10	50.1M	204,800	1.83×10^{-3}	3 / 3
80M	896	14	92.7M	327,680	2.03×10^{-3}	1 / 1
160M	1280	20	179.6M	524,288	1.56×10^{-3}	2 / 2 (middle block, $r = 4$)

and later loops read it while keeping a short window of their own, and find that this costs no quality. We test sharing only as a change made after training (Section 7).

3 Setup

3.1 Models

All language models are decoder-only transformers with eight unique layers of width w: pre-norm RMSNorm, SwiGLU feed-forward blocks (hidden size $\frac{8}{3}w$), rotary position embeddings, attention heads of dimension 64, no biases, untied input and output embeddings, a 16k BPE vocabulary trained on the training data, and a context of 1024 tokens. A *dense* model runs its eight layers once. A *middle-block* looped model runs two layers once (the prelude), then the middle four layers r times (the core), then two layers once (the coda), so it executes $2 + 4r + 2$ layers; a *whole-stack* looped model runs all eight layers r times. Each loop starts from a linear map of the concatenated state and prelude output (input injection, $2w^2$ parameters), the initial state is zero, and the output of every residual branch is scaled by $(8/12)^{-1/2}$ (eight layers against a 12-layer reference), divided by r inside the core, so that the best learning rate does not move with r (Wang et al., 2026a). Backpropagation runs through all loops or, for *truncated* cells, through the last $r/2$ loops only. Table 1 lists the sizes.

3.2 Data, training and the grid

Training uses one epoch of FineWeb-Edu sample-10BT (Penedo et al., 2024) (10.4B tokens) in a fixed order for every run. The main validation set is 50M held-out FineWeb-Edu tokens; three further sets of 20M tokens each come from SlimPajama, FineMath and GitHub code. We use AdamW ($\beta = (0.9, 0.95)$, weight decay 0.1, gradient clipping 1.0) and a warmup–stable–decay schedule with cooldown branches (Hägele et al., 2024): one constant-rate trunk per run, from which cooldowns of 20% of the budget, decaying linearly to zero, branch off at several budgets. A budget is given in training tokens per parameter, written $10N$ for ten tokens per parameter. Looped runs have budgets of $10N$, $20N$ and $40N$; dense runs $5N$ to $80N$ ($40N$ at 160M). Each looped size has ten cells: middle block with $r = 2$, $r = 4$ and $r = 8$, each with full backpropagation and, for $r = 4$ and 8, truncated; and the same five for the whole stack. With one dense model per size, three seeds at 10M–40M and one at 80M, and a 160M dense reference, the pre-registered grid has 111 runs. Seeds change the initialisation, not the data order.

Extensions, all reported as exploratory, add a 160M middle block looped 4 times and a second 160M dense seed, continue the 10M middle blocks looped 4 and 8 times to $80N$ and $160N$ (and the 10M dense model to $160N$) and the 20M ones to $80N$, train 20M models with a random loop count, and probe trained models loop by loop. Two batches of supplementary experiments, S1–S3 and S4–S9, ran under plans frozen before their runs; they re-tune learning rates (S1, S2), measure inference cost (S3) and KV sharing (S6), continue dense models to $640N$ (S4), change the data order (S7), and add chain-of-thought, pause-token, stopping-rule and training-regime experiments on the synthetic task (S5, S8, S9). Everything ran on four RTX 4090 GPUs, about 19 days of wall-clock time in all.

3.3 Measurement

Losses are in nats per token on the main validation set. Between initialisation seeds at 20M the loss varies by 0.015–0.024 nats; a difference of 0.01 nats between runs that share a seed is small but resolvable, one between unpaired runs is not.

The pre-registered law. The effective size of a looped model is $N_{\text{eff}} = N_{\text{once}} + r^{\varphi} N_{\text{rec}}$ (Schwethelm et al., 2026), where N_{once} counts the parameters run once and N_{rec} those in the loop; unlike Schwethelm et al. (2026), who count non-embedding parameters only, we include the output head in N_{once}, as in N. A dense "ruler" $L(N, D) = E + AN^{-\alpha} + BD^{-\beta}$ is fitted to the dense runs (Hoffmann et al., 2022; Besiroglu et al., 2024) with a Huber loss in log space ($\delta = 10^{-3}$), 500 random starts and 1000 block-bootstrap resamples; with the ruler fixed, φ is fitted per cell. A joint fit of the ruler and all φ is the pre-registered secondary analysis.

Without a fitted law. Three quantities need no ruler. The *gap* is the looped minus the dense loss at equal parameters and tokens, same seed. The *dense size of equal loss* $N^{\star}$ reads every measured dense size at the looped model's number of training tokens and interpolates $\log N$ linearly in the loss between them, which gives a multiplier $M = N^{\star}/N$ and a local exponent φ_{loc} defined by $N^{\star} = N_{\text{once}} + r^{\varphi_{\text{loc}}} N_{\text{rec}}$. The *equal-compute gap* converts a looped run's training FLOPs (counted over executed layers, including attention and the output head) into the tokens a dense model of the same size could have seen for them, and reads the dense loss there off the measured dense curve; where that point lies beyond the measured curve, two extrapolations bound it (Section 4.3).

3.4 Hypotheses and labels

The pre-registration, frozen on 2026-09-23 before any training run, states four hypotheses with decision rules: **H1** truncated backpropagation lowers φ, under both an equal-token and an equal-FLOP accounting; **H2** a looped middle block has a higher φ than a looped whole stack; **H3** φ saturates in r, so that a geometric form $N_{\text{eff}} = N_{\text{once}} + N_{\text{rec}}(1 - \rho^{r})/(1 - \rho)$ extrapolates better than r^{φ} to a held-out size in both directions; and **H4** under a training-compute budget the best r is small, under a deployment-compute budget larger. A factor counts only if its effect exceeds both the bootstrap interval and the seed spread. Throughout, results are labelled *pre-registered* (hypothesis and rule frozen before training), *pre-specified* (written in a plan frozen before the runs it describes: the synthetic design on 2026-10-09 and the two supplementary plans on 2026-10-13 and 2026-10-15) or *exploratory* (chosen after seeing data). Every departure from a frozen document is dated in the deviation log.

4 What a loop is worth on text

4.1 The pre-registered verdicts

The dense ruler, fitted to the 54 dense results of the grid, puts its irreducible loss at the lower bound of the fit ($E = 0.050$) and its parameter exponent at $\alpha = 0.054$, far below the usual value of about 0.3; five sizes cannot pin down E, A and α separately. The joint fit over all 352 results gives $\alpha = 0.267$ and larger φ. Table 2 gives φ per cell and the verdicts *(pre-registered)*.

Two observations from the grid matter for what follows. First, truncated backpropagation, predicted to hurt, helped whole-stack loops: at 20M a whole stack looped 4 times reached 3.646 with truncation and 3.783 with full backpropagation at $10N$ (three-seed means), and at 10M the only looped cells better than the dense model on average over the three budgets were whole-stack loops with $r = 8$. Second, truncation and whole-stack loops brought instability: 30 of the 111 runs had at least one loss spike, 25 of them with truncated backpropagation (16 of these whole-stack loops) and the other five whole-stack loops with $r = 8$; one run collapsed. Halving the learning rate removed the spikes in a 10M follow-up (Appendix B).

Table 2: Pre-registered results. φ per cell on the main validation set, equal-token accounting, N including the output head; primary analysis (ruler fitted first) with 95% block-bootstrap intervals, and the secondary joint fit. Below: the four hypotheses and their verdicts under the frozen decision rules.

Cell	φ, primary [95% interval]	φ, joint fit
Middle block, full backpropagation	0.13 [0.02, 0.27]	0.41
Middle block, truncated	0.00 [0.00, 0.17]	0.29
Whole stack, full backpropagation	0.00 [0.00, 0.16]	0.00
Whole stack, truncated	0.17 [0.10, 0.20]	0.29

Hypothesis	Verdict
H1: truncation lowers φ	Not supported. Middle block 0.00 vs 0.13 (predicted direction, intervals overlap); whole stack 0.17 vs 0.00 (opposite direction). The equal-FLOP accounting agrees.
H2: middle block > whole stack	Depends on the measure. φ is mixed; raw losses at matched executed depth favour the middle block in 24 of 24 comparisons, by 0.03–0.19 nats, less at larger sizes.
H3: φ saturates in r	Not supported. The geometric form predicts 80M from 10M–40M better (0.0085 vs 0.0096 rmse) and 10M from 20M–80M slightly worse (0.0203 vs 0.0200); the data do not separate the two forms.
H4: best r small under training, larger under deployment compute	First half holds trivially (the dense model, $r = 1$, is optimal under training compute); second half not supported (it is optimal under deployment compute too).

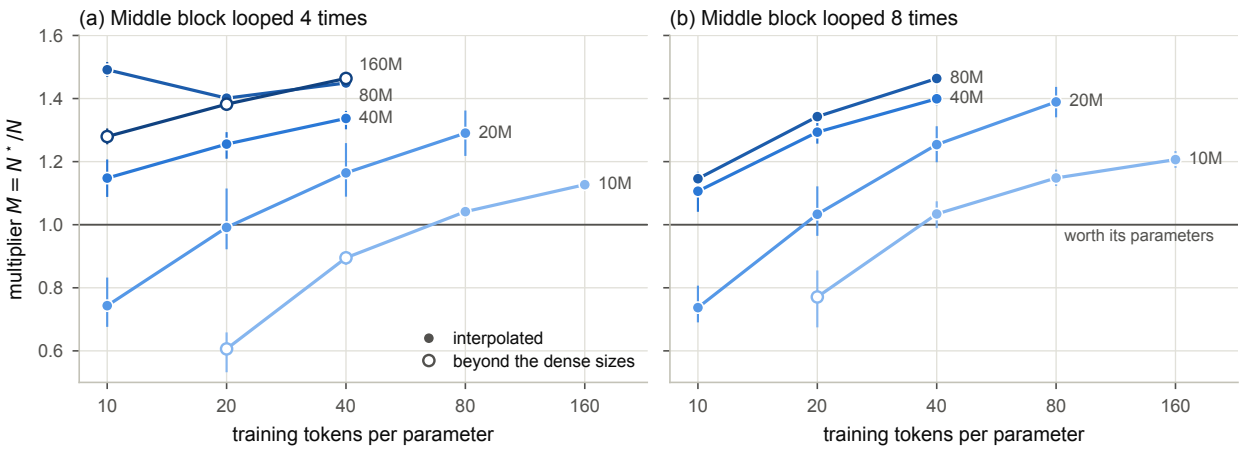


Figure 2: What a parameter of a looped middle block is worth, measured without a fitted law: the multiplier $M = N^\star/N$, where $N^\star$ is the size of the dense model that reaches the same loss on the same number of training tokens (interpolated between the measured dense sizes, including the dense models continued in S4; open markers: beyond the largest or smallest dense size by at most a factor of two). $M = 1$: the looped model is as good as a dense model of its own size. Bars: 95% intervals from resampling seeds (80M has one seed).

4.2 Why φ is not identified, and what to measure instead

φ converts a loss into effective parameters through the ruler, and the ruler is weakly identified: with the primary fit φ is 0–0.17 per cell, with the joint fit 0–0.41. Its trend across sizes is robust, though. With the ruler fixed, the middle block looped 4 times has $\varphi = -0.32$, 0.19, 0.39, 0.52 and 0.65 from 10M to 160M *(exploratory)*. A single φ per cell averages over very different regimes.

The dense runs themselves can serve as the ruler. For each looped model we read the loss of every measured dense size at the looped model's number of training tokens and interpolate the dense size $N^\star$ with the same loss (Figure 2, *exploratory*). The multiplier $M = N^\star/N$ of the middle block looped 4 times rises from 0.6–1.0 at 10–20M with 10–20 tokens per parameter to 1.26–1.46 at 40M–160M with 20–40; the 160M values lie beyond the largest dense model, by less than a factor of two. Counting N without the output head, as Schwethelm et al. (2026) do, the local exponent there is $\varphi_{\text{loc}} = 0.32$–$0.50$, close to their $\varphi = 0.46$ (0.35–0.52 with the head). A least-squares plane in $\log_2 M$ gives a factor of 1.21 per doubling of model size (95% interval 1.13–1.30) and 1.14 per doubling of tokens per parameter (1.05–1.24; 1.16 with 8 loops); the product term is indistinguishable from zero. The loop breaks even ($M = 1$) at about 35M parameters with 20 tokens per parameter and about 12M with 80. Eight loops add almost nothing over four at these sizes.

Published equal-parameter results restate to a similar range in comparable settings, and to higher values with far more tokens per parameter. Where a paper reports a non-looped ladder, we interpolate it in the same way. Parcae's models looped 8 times are worth 1.31–1.46 dense parameters per parameter at 140M–770M (Prairie et al., 2026), against a ladder trained on the same tokens per parameter, which understates the value. Ouro's models looped 4 times are worth 1.23 and 1.36 at 134M and 374M and less than their parameters at 53M, measured by mean benchmark accuracy read off a plot (Zhu et al., 2025, App. D.1). The two 12-layer looped models of Saunshi et al. (2025), about 0.5B parameters looped 2 and 4 times, are worth 1.26 and 1.36, and their shallower blocks looped 3, 4 and 6 times 1.47–1.95, at about 750–1500 tokens per parameter. The law of Schwethelm et al. (2026) implies 1.25, 1.45 and 1.56 for 2, 4 and 8 loops in their iso-depth layouts. These studies differ from ours in size, data and recipe, so the agreement is a consistency check, not a pooled estimate. They barely sample models below 40M trained on 20 or fewer tokens per parameter, where we find a loop worth at most about its parameters. That the value rises with tokens per parameter at equal parameters is an interaction that effective-parameter laws whose loop gain depends on the loop count (and, for mixtures of experts, on the sparsity) but not on the training tokens (Schwethelm et al., 2026; Chen et al., 2026a) do not capture; it agrees with the recommendation of Prairie et al. (2026) to increase looping and data in tandem, and with the finding of Chen et al. (2026b) that with repeated data the best loop count grows with compute.

4.3 At equal training compute, the dense model wins by less and less

At equal parameters, a middle block looped 2, 4 or 8 times costs 1.4–1.5, 2.2–2.4 or 3.7–4.1 times the training FLOPs per token of the dense model. For an equal-compute comparison we give the dense model the tokens it could have seen with the looped run's compute and read its loss off the measured dense curve. Of the 320 looped results of the grid and its extensions, 221 could be compared this way with the original dense runs; the dense model won all of them, by at least 0.016 nats. The other 99 needed the dense loss beyond its measured curve. Two extrapolations bracket it: in a backtest that drops the last budgets of every measured dense curve, extending the last segment predicts a dense loss 0.023–0.036 nats too low one doubling out (it flatters the dense model), and a power law fitted to the whole curve one 0.006–0.027 too high (it flatters looping). With the power law, looping came out ahead in 6 of the 99, by at most 0.021 nats, all at the largest data per parameter.

We then continued the 10M dense models to $640N$ and the 20M ones to $320N$ (supplementary experiment S4, *pre-specified*), which turns 37 of the extrapolated comparisons into measured ones (Figure 3, Table 3). The dense model wins all 258 measured comparisons, and the measured gaps fall about halfway between the two extrapolations. These comparisons are not independent: they share dense curves, trunks and seeds. The margin shrinks with data per parameter: for the 10M middle block looped 4 times it is $+0.119$ at $40N$, $+0.062$ at $80N$ and $+0.016$ at $160N$. Trained at its own learning rate (0.35 times the dense rate, found in S1), the same model reaches parity at $160N$ (-0.011 and $+0.005$ in two seeds), the only measured equal-compute comparison in this study that is not a dense win. The other 62 comparisons still need an extrapolated dense loss: 43 at 40M–160M, and 19 at 10M–20M, almost all for the third seed, whose dense curve ends at $80N$. At 160M and $40N$ the bracket reaches from $+0.004$ to $+0.043$. Within the measured range, loops trade compute for parameters and data; they do not save compute.

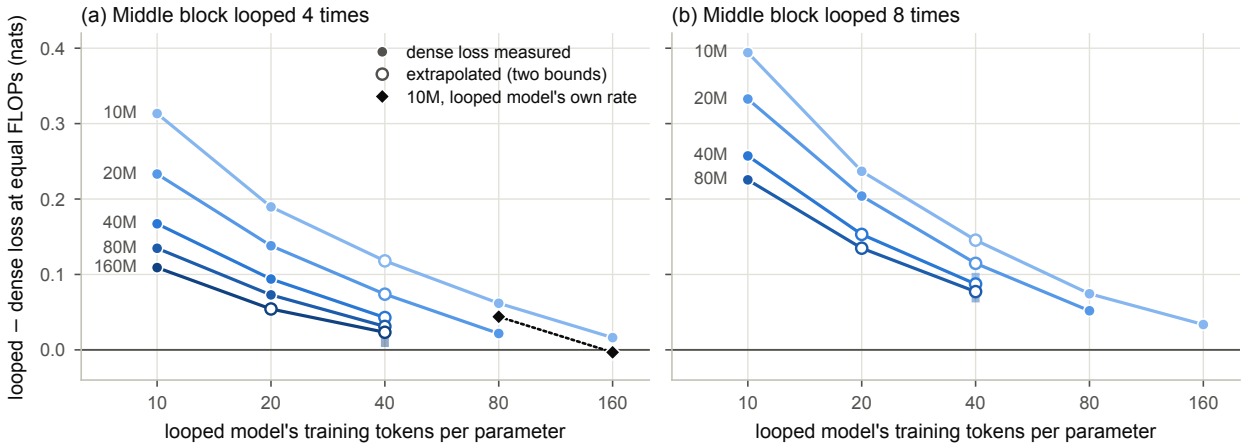


Figure 3: Equal training compute: looped minus dense loss when the dense model of the same size is trained on the tokens the looped run's compute would buy (positive: dense better). Filled markers: dense loss measured, including the dense models continued to $640N$ (10M) and $320N$ (20M) in S4. Open markers with bars: the dense loss is extrapolated; the bar spans the two bounds (power law to last segment). Diamonds: the 10M model looped 4 times trained at its own learning rate (0.35 times the dense rate; mean of two seeds, bars: seed range).

Table 3: The equal-compute comparisons at the largest looped budgets (S4; looped minus dense loss on the main validation set, means over two seeds; negative: looped better). At equal compute the dense model is trained on 2.2–3.8 times the tokens.

	Training FLOPs	Looped − dense loss	
Looped cell (size, tokens per parameter)	vs dense	equal parameters and tokens	equal compute
Middle block, $r = 4$ (10M, $160N$)	2.2×	−0.033	+0.016
Middle block, $r = 8$ (10M, $160N$)	3.7×	−0.051	+0.034
Middle block, $r = 4$ (20M, $80N$)	2.2×	−0.051	+0.022
Middle block, $r = 8$ (20M, $80N$)	3.8×	−0.064	+0.052
Middle block, $r = 4$, own rate (10M, $160N$)	2.2×	—	−0.011, +0.005 (seeds)

4.4 Learning rates

All looped language models use the dense models' learning rates, after a check of ten looped cells at 20M and $10N$, with its decision rule written before its runs, found no better one. Because the check covered neither longer budgets nor other sizes, we re-tuned both the dense model and the middle block looped 4 times at 10M, 20M and 40M (supplementary experiment S1, *pre-specified*): 0.5 to 1.41 times the dense rate, plus 0.35 times where the best rate lay at the lowest one tried, at $10N$–$40N$ (at 10M also $80N$ and $160N$), with one seed (two at 20M). The dense model's best rate stayed at 1×, except at 40M, where 0.71× was better by at most 0.004 nats. At 20M–40M the shared rate understated the looped model's advantage by at most 0.005 nats; at 10M the looped model preferred 0.5× and below from $20N$ on, worth about 0.02 nats. With each model at its best rate, the gap at $40N$ is −0.005, −0.031 and −0.055 nats from 10M to 40M (shared rate, same seeds: +0.015, −0.031, −0.050). The size trend is somewhat smaller but holds by the criterion of the plan (the improvement from 10M to 40M at the best rates exceeds half of that at the shared rate); the data trend at 10M is larger (+0.082 at $10N$ to −0.059 at $160N$). At equal compute the dense model won all 11 comparisons of the re-tuned runs, but the one at 10M and $160N$ needed an extrapolated dense loss; measured in S4, it is the parity reported above (Appendix D).

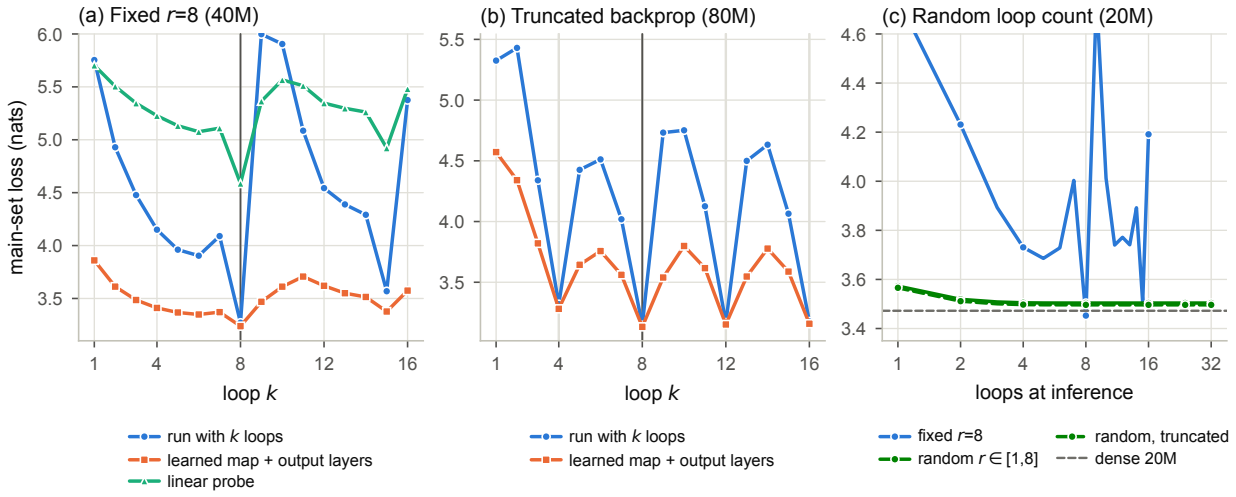


Figure 4: What the looped block computes on text (main-set loss of trunk checkpoints at $32N$; values above the top of a panel are clipped). **(a)** Fixed loop count: a 40M middle block trained with 8 loops (mean of three seeds), read after each loop k by running the model with k loops, by a learned map in front of the output layers, and by a learned linear head. The model works only at its training loop count. **(b)** Truncated backpropagation (80M whole stack, $r = 8$, gradients through the last 4 loops): the state returns to the same point every four loops. **(c)** A random loop count during training (20M middle block, r drawn from 1–8 at every step) contracts to a fixed point: flat from about four loops on, no better than the dense model, with or without truncation.

4.5 Where the gain lands, and what the seeds measure

Per token, the gain on web text is a small, broad shift: about half of the tokens improve, and the gain is not concentrated on hard tokens beyond what two dense seeds differ by *(exploratory)*. On math and code text the gain is larger, on average about 3 and 7 times the gain on the main set, but the seed-to-seed spread on those sets is 10–100 times larger as well. Supplementary experiment S7 *(pre-specified)* traced that spread to the data order: at 20M, training the same initialisation on three disjoint parts of the data stream moves the main-set loss by only 0.003–0.011 nats, 0.1–0.5 times the spread between initialisation seeds, but moves the code-set loss by 0.21–0.69 nats, up to 7 times that spread, and changes the sign of the looped-minus-dense gap there. Which documents a run happens to see decides its out-of-distribution loss at this size, so we draw no conclusions from differences on those sets.

5 What the looped block computes on text

If the loop iterated towards an answer, the state after each loop would carry a better and better prediction, and running more loops at inference would help. We probed trained models loop by loop *(exploratory)*: on trunk checkpoints at $32N$, we read the state after each loop k with the model's own output layers (the logit lens of nostalgebraist, 2020, which here is the same as running the model with k loops), with a learned linear $w \times w$ map in front of the output layers (in the spirit of the tuned lens of Belrose et al., 2023, but trained on the next token rather than distilled from the model's own output), and with a learned linear head; the maps are trained on 10M unseen training tokens and evaluated on 2M tokens of the main validation set. Figure 4 shows the three training regimes.

A fixed loop count gives a fixed-length program. Trained with 8 loops, the model works only at 8 (Figure 4a). With a learned map, the state after loop 4 already reads out within 0.1–0.2 nats of the final loss, and loops 5–7 add little; the last loop is a large non-linear move that rewrites the state into what the output layers expect, and 37–44% of the tokens settle their prediction only there. Running fewer or more loops breaks the model. This held in all three seeds at 20M and 40M and at 80M (one seed).

Truncated backpropagation gives a cycle. With gradients only through the last four loops, the whole stack learns a periodic cycle (Figure 4b): the state returns to almost the same point every four loops, so the model works at 4, 8, 12 or 16 loops and gains nothing from more. The same minima at 4, 8 and 12 loops appear at 20M and in both probed 40M seeds, one of which also dips at every second loop.

A random loop count gives a fixed point. Drawing the loop count from 1–8 at every step, as Huginn does with a different distribution (Geiping et al., 2025), gives a block that contracts to a fixed point: the loss is flat from about four loops to 32 (Figure 4c; one seed at 20M, with full and with truncated backpropagation). The model is robust to the loop count, but at its nominal 8 loops it is 0.05 nats worse than fixed-count training and no better than the dense model; random-loop training erased the advantage of looping at this size.

The learning rate shapes the iteration but never makes extra loops help. Lower learning rates give flatter curves over the inference loop count. In S1, on the middle block looped 4 times at 10M, 20M and 40M, the loss varies by 0.36–0.77 nats over 4–16 loops at 0.5 times the dense rate and by 3.4–4.6 nats at 1.41 times. At every rate, the better of 8 and 16 loops was worse than the training count of 4. On text, more loops than trained never helped.

6 Loops as extendable depth on a serial task

Predicting the next word of web text rarely needs many serial steps, so text gives the loop little reason to iterate. We therefore built a task that does, froze its design, its conditions and its predictions before the main runs, and ran it with the same architecture. Pilot runs then led to three changes before any formal run, all for learnability: the operations became lookup tables instead of modular arithmetic, a curriculum was added, and training was lengthened; the learning-rate protocol was fixed after the pilot (Appendix G).

6.1 The task and the conditions

A problem lists a start value and a sequence of table lookups, for example `25 ; LUT 7 ; LUT 3 ; LUT 19 ; ... =>`, where each `LUT k` is one of 32 fixed random functions on $\{0, \ldots, 96\}$. The model must output the final value in a single forward pass; the loss is taken at the answer position only. Two lookups compose into one of 1024 arbitrary functions with no compact form, so a model that skips steps must memorise rapidly growing tables of compositions; the task rewards computing the chain step by step, although, as for any associative composition, a shallower solution exists in principle (Liu et al., 2022). A prompt with d steps has $3d + 4$ tokens. Models are trained on 1–8 steps, sampled online with a curriculum that widens the range over the first half of training, for 32,000 steps of 512 problems, and are tested on 2000 fresh problems at every length from 1 to 24. Chains longer than 8 steps never occur in training; short problems do repeat, since there are only 3,104 one-step problems.

All models have width 256. The conditions are dense models of 8, 20 and 36 layers (6.5M, 16.1M and 29.0M parameters) and 8-layer looped models (6.6M parameters, a middle block of four layers looped r times) trained in four regimes: with $r = 2$, 4 or 8 loops; with a random loop count drawn from 1–8 at every step; and with as many loops as the chain has steps ("loops = steps", the regime of Fan et al. (2025)). The 20- and 36-layer dense models match the executed depth of the looped models with 4 and 8 loops, with 2.4 and 4.4 times their parameters. Each condition's learning rate was chosen by a sweep on in-range accuracy only, so that extrapolation played no part in the choice; the looped rates (1.25–2.5×10^{-4}) came out 2–8 times lower than the dense ones (5×10^{-4}–10^{-3}). Every condition has five seeds. The design fixed five predictions (P1–P5, Appendix E); a planned pointer-chasing control was dropped because no condition learned it under the chain-calibrated recipe.

6.2 Loops buy steps, and more loops extend them

At equal parameters, more loops solve longer chains: 4.2, 6.4, 8.0 and 9.0 steps for the dense model and 2, 4 and 8 loops (Table 4). Dense models of equal executed depth, with 2.4–4.4 times the parameters, solve

Table 4: The chain task. Largest solvable length: mean (min–max) over five seeds of the longest chain length up to which every length is solved with accuracy ≥ 0.9. "Own": at the training loop count (for loops = steps, d loops); "more": the best of the training and every larger inference loop count, including d loops, chosen per seed with the answers (for $r = 2$ including 4 and 8 loops). Chance is 0.01.

		Largest solvable length		Accuracy, own / more loops	
Model	Parameters	Own loops	More loops	12 steps	16 steps
Dense, 8 layers	6.5M	4.2 (4–5)	—	0.06	0.05
Dense, 20 layers	16.1M	6.4 (6–7)	—	0.04	0.03
Dense, 36 layers	29.0M	8.0 (8–8)	—	0.02	0.02
Looped, $r = 2$	6.6M	6.4 (6–7)	8.6 (6–11)	0.05 / 0.64	0.04 / 0.33
Looped, $r = 4$	6.6M	8.0 (8–8)	11.8 (10–15)	0.03 / 0.83	0.03 / 0.28
Looped, $r = 8$	6.6M	9.0 (9–9)	11.2 (11–12)	0.04 / 0.86	0.03 / 0.37
Looped, random loop count	6.6M	11.4 (10–13)	12.2 (10–15)	0.77 / 0.83	0.08 / 0.49
Looped, loops = steps	6.6M	12.8 (11–16)	—	0.96	0.37
Loops = steps + 0–4 extra loops (S9)	6.6M	11.6 (11–12)	—	0.90	0.31

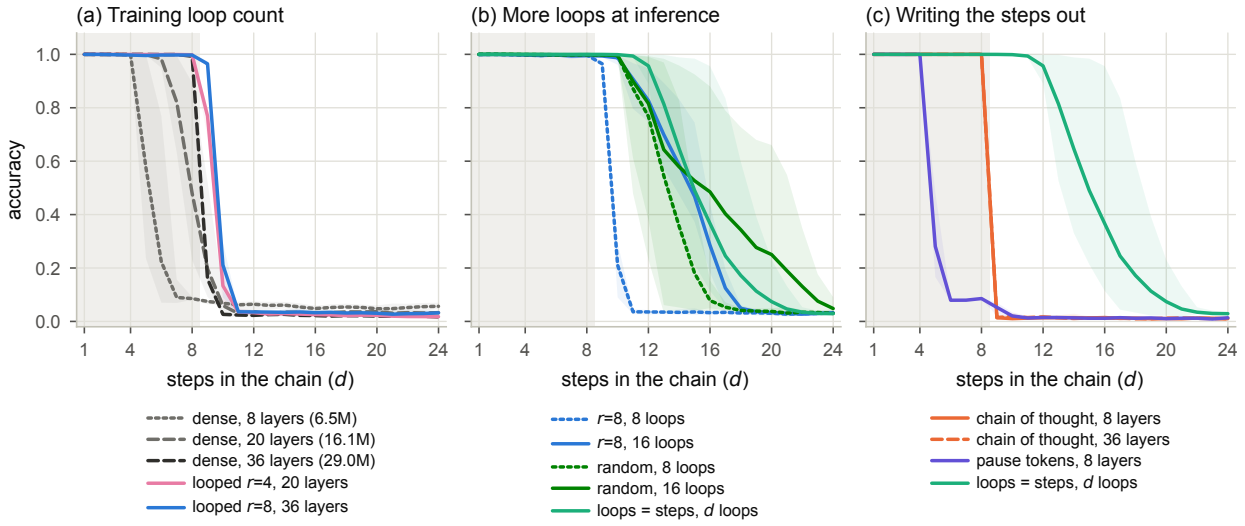


Figure 5: The chain task, trained on 1–8 steps (shaded); accuracy against chain length, mean over five seeds (band: range). **(a)** At the training loop count, looped models solve at least as far as dense models of the same executed depth, and all stop at the training range. **(b)** With more loops at inference, the looped models solve longer chains; the model trained with as many loops as steps, run with d loops, solves the longest chains at accuracy 0.9. **(c)** Dense models that write the intermediate values out are perfect within the training range and at chance beyond it; pause tokens add nothing (Section 6.6).

1–1.6 fewer steps (the 20-layer one did not fully learn the training range), and all dense models fail right after the training range: at 12 steps they score 0.02–0.06 (Figure 5a). So do the fixed-loop models at their training loop count (0.03–0.05). What the dense models lack is not depth but the option to run longer.

More loops at inference solve longer chains (Figure 5b). The model trained with 8 loops solves 9 steps at 8 loops and about 11 with 16 or 32; at 12 steps its accuracy rises from 0.04 to 0.86 with the best loop count of 12–32 chosen per seed (0.83 at 16 loops, on average). The gain is real but bounded: at 16 steps the best is 0.37, with a wide spread across seeds. The model trained with a random loop count gains as well (0.08 to 0.49 at 16 steps), against the frozen prediction that it would not. The model trained with loops = steps, run with as many loops as the chain has steps, solves the longest chains at accuracy 0.9 (12.8 steps) and beats 8 loops by 0.34–0.95 at every length from 10 to 16, as predicted, partly because the problem states its number of steps; at 16 steps the random-loop model with 16 loops is more accurate (0.49 against 0.37).

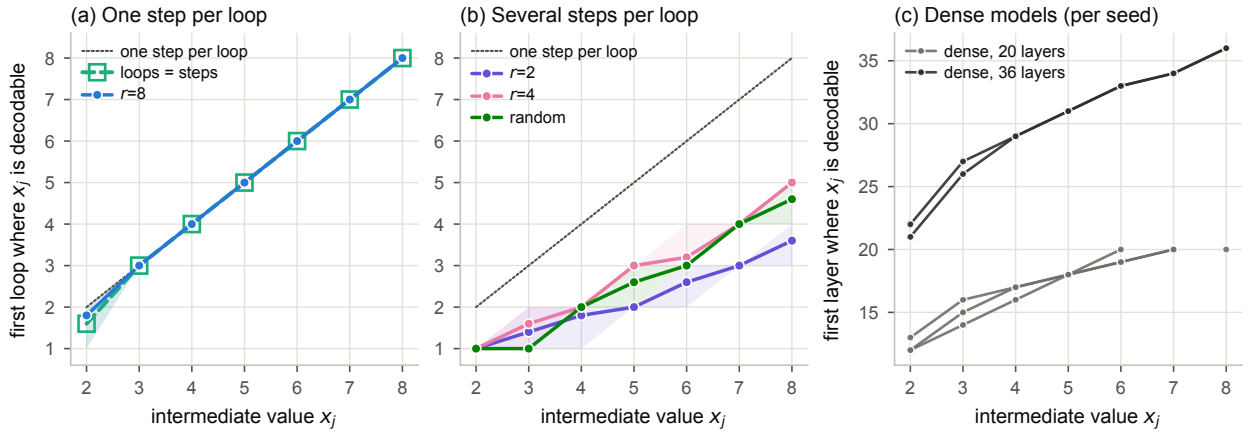


Figure 6: Where the intermediate values appear (probes on 8-step problems; a value counts as decodable at probe accuracy ≥ 0.5). **(a)** With a loop for every step, the chain advances one step per loop. **(b)** With fewer loops than steps, each loop advances several steps (mean over seeds; band: range). **(c)** Dense models compute the values late in their depth (one line per seed; a seed in which only the answer is decodable appears as a single point at the last layer, and the 20-layer seed with no decodable value is not shown).

6.3 One step per loop

For each intermediate value x_j we train a linear probe on the state after every loop, at the input positions of step j, on problems with 8 steps (Figure 6). The *frontier* is the first loop at which each value becomes decodable (probe accuracy ≥ 0.5), and *persistence* is the lowest probe accuracy a value reaches in the loops after that, averaged over the values. In the model trained with 8 loops, x_j first becomes decodable at about loop j (1.0–1.1 loops per step in all five seeds) and stays decodable afterwards (persistence 0.96 within its loops). The loops = steps model also computes one step per loop and keeps its values for its d loops (0.92), but loses them when run longer: it was never trained to keep looping after the last step, which is why it needs exactly d loops. Models with fewer loops than steps pack more steps into each loop, about 1.6 per loop with 4 loops and 2.4 with 2, the direction predicted by the budget law of Zhang et al. (2026). The dense models compute late (from about layer 21 of 36), and in 5 of the 10 seeds of the 20- and 36-layer models no intermediate value appears at the step positions at all: four show only the answer, at the last layer, and one shows nothing. Measured by the share of values that first become decodable at their own loop, the loops = steps and 8-loop models are diagonal (0.95 and 0.97) and the others are not (0.12).

6.4 The learning rate decides whether the loop learns the task

In the learning-rate sweep, the 8-loop model trained at 10^{-3}, a good rate for the dense models, worked only at exactly 8 loops and had no step-by-step frontier, like the language models of Section 5; but it had also learned the training range much worse (0.54 against 0.999 at 2.5×10^{-4}). Probes of all sweep checkpoints show the frontier at rates up to 5×10^{-4} and not at 10^{-3} or 2×10^{-3}, for 4 and 8 loops and the random loop count alike. We then trained the four looped conditions that extrapolated in the main runs (4 and 8 loops, random loop count, loops = steps; supplementary experiment S2, *pre-specified*) at 5×10^{-4} and 10^{-3} for the full 32,000 steps with three seeds. At 10^{-3}, 10 of 12 runs did not learn the training range in that time (accuracy 0.49–0.98); at 5×10^{-4}, 8 of 12 did, and those mostly extended about as far as the formal runs (8 loops: 2.7 steps beyond the training range against 3.2; loops = steps: 4.0 against 4.8), except the random loop count, whose two learned seeds extended 2 steps against 4.2. A high learning rate mainly keeps the looped model from learning the task in the given steps; when it learns the task, it mostly learns the same extendable algorithm.

Table 5: Stopping without the answer: accuracy on 12-step chains (seed means, reporting set). The best fixed loop count is chosen with the answers and is an upper reference, not a rule.

Model	Own loop count	Best fixed count	State change $< 2\%$	Confidence ≥ 0.99 (mean loops)
Looped, $r = 2$	0.05	0.62	0.45	0.70 (8.3)
Looped, $r = 4$	0.04	0.83	0.65	0.81 (9.2)
Looped, $r = 8$	0.04	0.86	0.71	0.84 (17)
Random loop count	0.77	0.83	0.73	0.83 (14)
Loops = steps (d loops)	0.96	0.96	0.03	0.76 (14), 6–7% lost in range
Loops = steps + 0–4 extra (S9)	0.90	0.94	0.52	0.93 (14), 0.998 kept in range

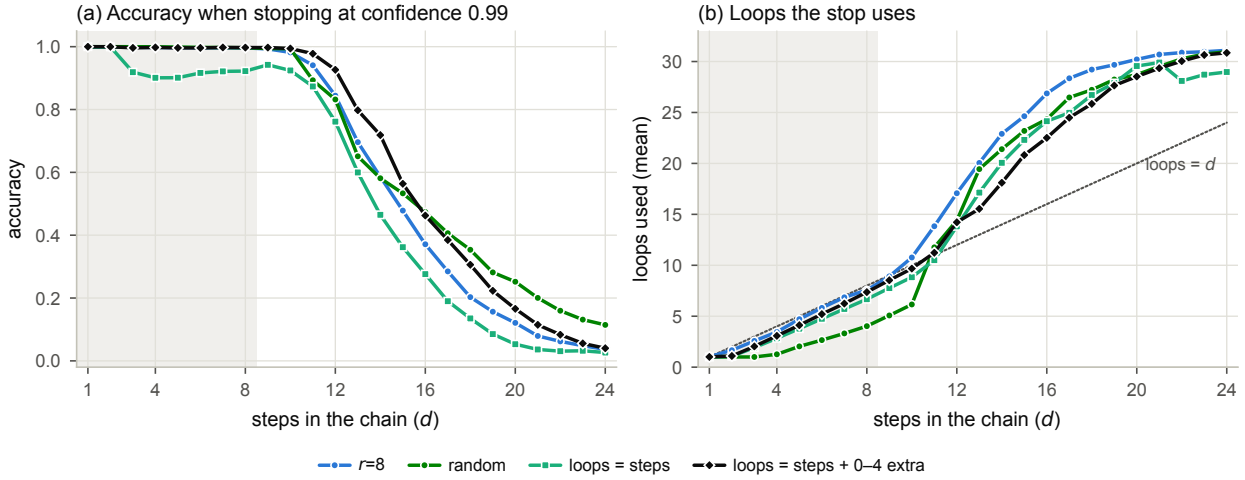


Figure 7: Stopping when the model's top probability reaches 0.99 (reporting set, mean over five seeds). **(a)** Accuracy. **(b)** Loops used: the stop runs few loops on short chains and many on long ones. The model trained with loops = steps loses part of the training range; trained with a few random extra loops, it keeps the training range and stops best of the four beyond it, up to 15 steps.

6.5 Knowing when to stop

Extra loops help only if the model can tell how many to run. The frozen design specified one answer-free rule: stop when the state at the answer position changes by less than 2% between loops. We compared it (supplementary experiment S8, *pre-specified*), on the same 25 checkpoints, with rules that stop when the prediction has not changed for m loops or when the model's top probability reaches a threshold τ (the early-stopping variant of the maximum-confidence rule of Fan et al., 2025), and with majority votes over loop counts; the rules and thresholds were fixed in the plan, selected on one problem set and reported on another. Table 5 and Figure 7 give the results.

The state rule recovers about 80% of the gain of the best fixed loop count at 12 steps for the models trained with 4 or 8 loops, without hurting the training range, but it cannot stop the loops = steps model, whose state keeps changing after d loops: it stops on a wrong loop at every length (0.03–0.08). The model's own confidence does better, and it is the best of the rules that stop in every condition. (The plan's selection rule, the highest accuracy over 9–16 steps on the selection set among rules that keep the in-range accuracy within 0.01 of the training loop count's, also admitted votes over loop counts and picked a vote for three of the five conditions; a vote always runs all 32 loops and only ties the best fixed count.) At a threshold of 0.99 the confidence rule recovers 97–98% of the best fixed count's gain at 12 steps for the models trained with 4 or 8 loops or a random count, using fewer loops than their training count on in-range problems, and it brings the loops = steps model to 0.76 at a cost of 6–7% of the in-range problems. Stopping when the prediction stops changing fires on stable wrong answers, because the fixed-loop models turn their read-out only in the last loop; votes over loop counts tie the best fixed count but never beat it, and they are not stopping rules.

Table 6: Loops against a written chain of thought and pause tokens (seed means). Inference cost counts layer passes over the tokens processed: a 12-step prompt has 40 tokens, and the chain-of-thought models generate 12 more with a KV cache.

Model	Parameters	1–8 steps	12 steps	16 steps	Layer passes (12 steps)
Dense, 8 layers, chain of thought	6.5M	1.000	0.014	0.015	416
Dense, 36 layers, chain of thought	29.0M	1.000	0.017	0.013	1872
Dense, 8 layers, d pause tokens	6.5M	0.566	0.015	0.012	416
Dense, 8 layers, answer only	6.5M	0.62	0.064	0.048	320
Looped, $r = 8$: 8 loops / 16 loops	6.6M	0.998	0.036 / 0.827	0.033 / 0.285	1440 / 2720
Looped, loops = steps, d loops	6.6M	1.000	0.957	0.366	2080

The models that extend furthest and the models that stop well thus came from different training regimes. We therefore trained the loops = steps model with 0–4 random extra loops after the last step (supplementary experiment S9, *pre-specified*). It keeps its values when run longer (persistence over 16 loops 0.04 $\rightarrow$ 0.91), extends about one step less (11.6 against 12.8), and stops by confidence at 0.93 on 12-step problems while keeping 0.998 of the training range, using fewer loops than d in range. By the frozen state rule its stop is seed-dependent (0.52: it works in two seeds and fails in two), an outcome that none of the three interpretations written into the plan in advance covered. Training it costs about 1.4 times the compute per step.

6.6 Against reasoning written out

We gave the same problems to dense models that write the intermediate values out as tokens before the answer, a chain of thought (Wei et al., 2022) learned from scratch as in scratchpad training (Nye et al., 2021), with 8 layers (the looped models' size) and 36 layers (their executed depth), and to an 8-layer model with d pause tokens before the answer (Goyal et al., 2024) (supplementary experiment S5, *pre-specified*); five seeds each, learning rates chosen as for the other conditions. Both chain-of-thought models scored 1.000 in range at every rate tried, so the protocol's tie-break gave them the lowest one.

Within the training range the chain of thought is perfect, 1.000 in every seed and at every learning rate tried, where the 8-layer model that answers directly solves 4 steps; and it costs a third of the looped model's inference compute at its training loop count (Table 6). Beyond the range it does not extend at all: 0.014 at 12 steps with 8 layers and 0.017 with 36, chance level, against 0.83–0.96 for the looped models on the same prompts (Figure 5c). A diagnostic *(exploratory)* shows where it breaks: as soon as the prompt holds more than 8 tables, the first wrong token is one of the first three written ones in 97–98% of the problems for the 8-layer model, and even when the correct earlier tokens are given, the tokens after the first two are mostly wrong (accuracy 0.04–0.10). The failure is one of generalising to longer inputs, a known weakness of transformers (Zhou et al., 2024; Cho et al., 2024), not an accumulation of errors along the chain. Because the prompt of this task grows with the number of steps, the comparison cannot say whether a written chain would extend on a task whose input length does not grow, and remedies for length generalisation were outside the plan. Pause tokens add nothing: the pause model solves the same 4 steps as the model without them.

So on this task the question "loops or tokens?" has two answers. For problems like the training ones, tokens: cheaper and perfect. For longer ones, loops: the only form of extra computation here that extended.

6.7 Against the pre-specified predictions

The frozen design made five predictions. P1 (more loops, more steps at equal parameters) holds in direction, but the 8-loop model did not reach the predicted 16 steps. P2 (dense models of equal depth at least as good) is not supported as stated, with small gaps of 1–1.6 steps in favour of the loop. P3 (inference-time behaviour) fails in its first two parts, in the direction that favours looping (fixed-loop models were not hurt by extra loops, and the random-loop model, predicted not to gain, gained), and holds in its third (the loops = steps model extends). P4 (only the loops = steps model computes along the diagonal) is half right, since the 8-loop model does as well. P5 (much larger gaps than on text) holds. Of the supplementary predictions, S5 held one of three, S8 one of five and S9 three of four (Appendix E).

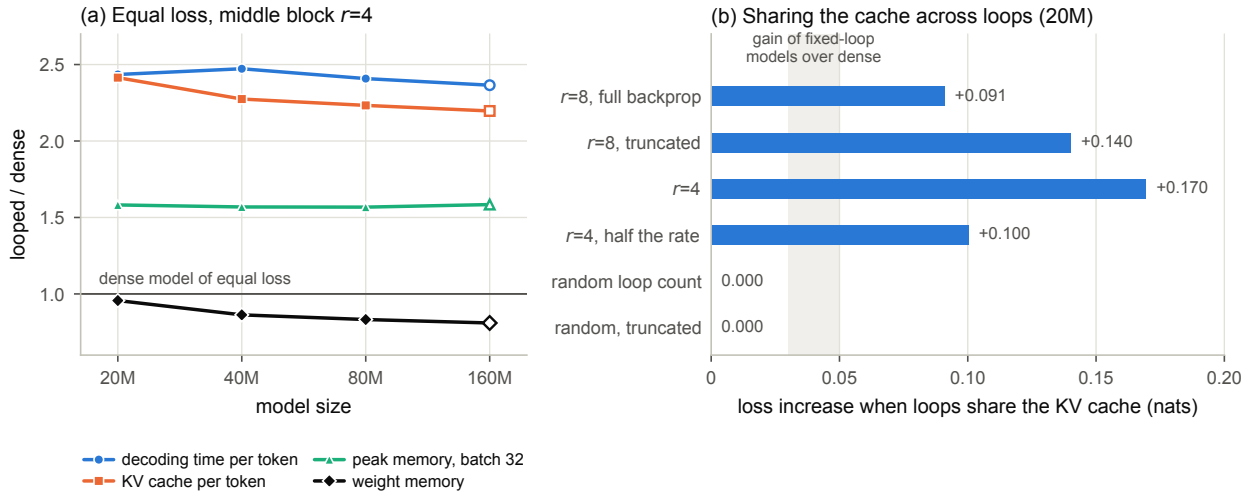


Figure 8: Deployment cost. **(a)** A middle block looped 4 times against the dense model of equal loss at $40N$: ratios of decoding time per token (batch 1), KV cache per token, peak memory at batch 32 (1k context) and weight memory (open markers: 160M lies beyond the largest dense model). **(b)** Sharing the KV cache across loops after training (20M, one cache slot per unique layer, written at a position's last loop): the loss increase on the main set; the shaded band is the fixed-loop models' gain over the dense model of the same size.

7 What loops cost at deployment

We measured inference on one RTX 4090 in bf16 with KV-cached decoding, one cache per executed layer, which is the computation the models were trained with (supplementary experiment S3, *pre-specified*). At equal parameters, a middle block looped 2, 4 or 8 times takes 1.5, 2.4–2.6 or 4.5–4.6 times the decoding time per token of the dense model. The fairer comparison is at equal loss: against the dense model that reaches the same loss at $40N$, interpolated between the dense sizes (Figure 8a; all cells in Appendix F).

At equal loss, the middle block looped 4 times needs 4%, 14%, 17% and 19% less weight memory than the dense model from 20M to 160M (at 160M the dense model of equal loss is extrapolated beyond the largest one trained), but 2.20–2.41 times the KV cache per token and 2.37–2.47 times the decoding time, and its peak memory at batch 32 is about 1.6 times that of the dense model. At batch 1 the larger KV cache cancels nearly all of the saving in weights. In this range, loops save parameters, not latency or memory; the saving would grow where weights dominate memory, for example one sequence with a short context in a much larger model, which we did not test.

Sharing the KV cache across loops would remove most of this cost. We applied terminal sharing, one cache slot per unique layer written at a position's last loop and read by every later loop, to trained 20M models of one seed (supplementary experiment S6, *pre-specified*; Figure 8b). Every model that gains from its loops at this size loses two to four times that gain: 0.09–0.17 nats against gains of 0.03–0.05 over the dense model, so with a shared cache it would fall below the dense model of the same size. Truncated backpropagation does not make the cache shareable here, unlike in Huang et al. (2026). The only models that share for free are those trained with a random loop count, whose block contracts to a fixed point (Section 5) and which gain nothing over the dense model. Among the fixed-loop models, a lower learning rate loses less (0.100 against 0.170 with 4 loops). The likely ways to get sharing are training towards convergence, or training with a shared cache; Monea et al. (2026) train with a different scheme, in which only the first loop writes a cache and later loops read it while keeping a short window of their own, and find that it costs no quality. We tested neither.

8 Discussion: when is recurrent depth test-time compute?

The same architecture behaves as a parameter-efficient model on text and as an extendable serial computer on the chain task. We offer a reading of the difference. It is an interpretation of two settings, not a tested law: in this reading, recurrent depth works as test-time compute when three conditions hold.

Headroom. The model's errors must come from too few serial steps rather than from missing knowledge. On the chain task the dense models know every table and fail only on depth; one more loop is one more step. On web text at these sizes we found no sign of such errors: the per-token gain is a diffuse shift rather than a concentrated one, and extra loops at inference never helped, at any learning rate (Sections 4.5 and 5). This is the hardest of the three conditions to measure independently, since here it is inferred from where extra loops do and do not help. The larger gains on math and code text hint at more headroom there, but at this size those sets are dominated by which documents a run happened to see.

Extendable iteration. The loop must be able to run longer without destroying its state. Which kind of iteration a loop learns is decided by its training more than by the architecture. On text the same block becomes a fixed-length program, a periodic cycle or a fixed point, depending on the loop schedule (Section 5). On the chain task the loop schedule decides whether the learned algorithm survives extra loops: models trained with as many loops as steps lose their values when run longer, and a few random extra loops in training fix that; the learning rate decides whether the task is learned at all (Sections 6.3–6.5). A fixed point is extendable but, on text, computes nothing more; a fixed-length program computes but cannot be extended.

A stopping signal. Extra loops are only useful if the model can tell how many to run. On the chain task the model's own confidence is such a signal for most regimes, and training a few loops past the last step makes the regime that extends furthest stoppable as well (Section 6.5). We did not test stopping rules on text, where extra loops did not help.

The chain task meets all three conditions once the model is trained for them; web text at these sizes showed no sign of headroom, and its one extendable regime, the fixed point, gained nothing from running longer. Two follow-ups we did not run would test this reading: a serial task whose input length does not grow with the number of steps, which would also decide whether a written chain of thought extends when the prompt does not get longer; and training on text that needs more serial steps, such as mathematics and code, compared at equal compute.

The value of a loop is not a constant of the architecture. Even where the loop does not iterate, it buys something on text, and what it buys depends on size and data: at 10–20M with up to 20 tokens per parameter a loop is worth at most about its parameters, at 40M–160M with 20–40 tokens per parameter about 1.3–1.5 times them. Effective-parameter laws whose loop gain depends on the loop count but not on model size or training tokens describe the second regime well and the first not at all, and the single pre-registered φ averaged over both. Studies at one scale will undersell or oversell the loop depending on where they sit.

Practical guidance. Within the range we measured:

- Loop when parameters or data are the constraint, not compute or latency. The advantage at equal parameters grows with model size and tokens per parameter; at equal training compute a dense model was better in every measured comparison but one, a tie at the smallest size and the most data with the looped model's own learning rate; at deployment the loop pays its executed layers in decoding time and KV cache.
- Loop a middle block with full backpropagation, start with four loops on text, and judge at the target scale: at 40M–80M eight loops did about as well as four at about 1.75 times the training compute, and small-scale ablations undersell the loop.

- Tune the looped model's learning rate separately, especially at small scale and on tasks that need serial steps: at a learning rate that suits the dense model, most looped runs did not learn the chain task in the given steps.

- Do not expect test-time scaling on text from standard training. To see whether a model can use more loops, sweep the loop count at inference; to see whether it computes step by step even when it cannot, probe it.

- If a model must decide by itself how long to run, its confidence is a good stopping signal, and training a few loops past the last step makes the stop reliable. On our task, a written chain of thought was the cheaper choice within the training range and failed beyond it; that failure came from longer prompts, so it says little about tasks whose input length is fixed.

9 Limitations

- **Scale and evaluation.** Models of 10M–160M parameters with a 16k vocabulary, far below the sizes at which looped language models are deployed, evaluated by validation loss only, without downstream benchmarks. The trends with size may bend; there are five sizes, one seed at 80M and one looped cell at 160M.

- **Data.** One source, FineWeb-Edu, and one epoch. Data-limited or repeated-data regimes, where loops may matter more (Chen et al., 2026b; Gülbahar et al., 2026), are not covered. Seeds change only the initialisation, except in the data-order experiment.

- **Equal compute.** 62 of the 320 equal-compute comparisons rest on an extrapolated dense loss, bracketed by two methods (Section 4.3), and the measured comparisons are not independent of each other.

- **Learning rates.** The dense rates come from a sweep at one budget; looped rates were re-tuned for one cell at 10M–40M with one seed (two at 20M), and the other cells and 80M–160M use the dense rates, which understated the looped model by up to 0.024 nats at 10M. On the synthetic task, several conditions tied at the top of the learning-rate criterion and the lowest rate was taken, while extrapolation is sensitive to the rate.

- **Single seeds** at 80M, for the random-loop language models, and for the KV-sharing test.

- **One synthetic task family.** The serial-task evidence comes from chains of table lookups and 6.6M-parameter looped models; extrapolation length varies widely across seeds. A pointer-chasing control was not learned and was dropped, and no dense control of equal executed depth was trained on text.

- **The comparison with a chain of thought** covers one task, models of 6.5M–29M parameters and one training recipe. Its prompt grows with the number of steps, so the chain of thought's failure beyond the training range is a failure to generalise to longer inputs, which the looped models handle on the same prompts; whether a written chain extends on a task whose input length does not grow was not tested. The chain-of-thought models got no help with length generalisation, and continuous thoughts (Hao et al., 2025) were not tried.

- **Deployment** was measured on one GPU type, for models of at most 160M parameters, with one KV cache per executed layer; KV sharing was tested only as a change after training, at 20M.

- **Exploratory analyses.** The multiplier without a fitted law, the equal-compute bounds, the probes and the stopping comparison beyond the frozen rule were chosen after seeing data and are labelled as such; the pre-registered verdicts stand on their own.

10 Pre-registration, deviations and reproducibility

The pre-registration, with the hypotheses, the grid, the fitting protocol and the decision rules, was frozen on 2026-09-23 before any training run, and its SHA-256 was committed to the repository the same day. The synthetic design and its predictions were frozen on 2026-10-09, and the two supplementary plans on 2026-10-13 and 2026-10-15, each before its runs. Every later change is dated in an append-only deviation log, together with its reason and its effect on the conclusions; Appendix G summarises them. The most consequential were made before any comparison between conditions: the synthetic task moved from modular arithmetic to lookup tables, and gained a curriculum, because modular operations were not learnable at this size. Two reviews of the results (2026-10-14 and 2026-10-15) found five errors and then about 35 errors and overstatements, among them a verdict judged with a quantity the frozen design did not specify; all were corrected and logged before this paper was written. One evaluation that the frozen design specified, the stopping rule of the loops = steps model, was computed but not reported until the second review.

Code, configurations, the per-run results and training logs of all 232 language-model runs and 135 synthetic-task runs, the fits, and the scripts that make every table and figure are available at `https://github.com/bethehand/looped-scaling`. Fits and figures run on a laptop CPU; the model checkpoints (485 GB) are not included.

Use of AI assistance. The study was carried out with the help of an AI assistant (Claude, by Anthropic), which drafted code, analyses, documents and this text under the direction of the author. Every decision recorded in the pre-registration and the deviation log, and responsibility for the content, lie with the author. Every number in the paper is computed by scripts in the repository.

References

Cem Anil, Ashwini Pokle, Kaiqu Liang, Johannes Treutlein, Yuhuai Wu, Shaojie Bai, Zico Kolter, and Roger Grosse. Path Independent Equilibrium Models Can Better Exploit Test-Time Computation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2211.09961.

Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA. In *International Conference on Learning Representations (ICLR)*, 2025a. arXiv:2410.20672.

Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. *arXiv preprint arXiv:2507.10524*, 2025b.

Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep Equilibrium Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1909.01377.

Andrea Banino, Jan Balaguer, and Charles Blundell. PonderNet: Learning to Ponder. In *8th ICML Workshop on Automated Machine Learning*, 2021. arXiv:2107.05407.

Arpit Bansal, Avi Schwarzschild, Eitan Borgnia, Zeyad Emam, Furong Huang, Micah Goldblum, and Tom Goldstein. End-to-end Algorithm Synthesis with Recurrent Networks: Logical Extrapolation Without Overthinking. *arXiv preprint arXiv:2202.05826*, 2022.

Nora Belrose, Igor Ostrovsky, Lev McKinney, Zach Furman, Logan Smith, Danny Halawi, Stella Biderman, and Jacob Steinhardt. Eliciting Latent Predictions from Transformers with the Tuned Lens. *arXiv preprint arXiv:2303.08112*, 2023.

Tamay Besiroglu, Ege Erdil, Matthew Barnett, and Josh You. Chinchilla Scaling: A replication attempt. *arXiv preprint arXiv:2404.10102*, 2024.

Yanbei Chen, Anirudh Goyal, and Raghuraman Krishnamoorthi. Scaling Laws for Looped Mixture of Experts. *arXiv preprint arXiv:2609.40316*, 2026a.

Zixi Chen, Akshay Vegesna, Samip Dahal, and Andrew Gordon Wilson. How Model Growth, Recursion, and Boundary Operators Influence Scaling Exponents. *arXiv preprint arXiv:2609.19107*, 2026b.

Hanseul Cho, Jaeyoung Cha, Pranjal Awasthi, Srinadh Bhojanapalli, Anupam Gupta, and Chulhee Yun. Position Coupling: Improving Length Generalization of Arithmetic Transformers Using Task Structure. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.20671.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal Transformers. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1807.03819.

Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. Looped Transformers for Length Generalization. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2409.15647.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. *arXiv preprint arXiv:2502.05171*, 2025.

Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training Language Models With Pause Tokens. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.02226.

Alex Graves. Adaptive Computation Time for Recurrent Neural Networks. *arXiv preprint arXiv:1603.08983*, 2016.

Serdar Gülbahar, Lukas Edman, and Alexander Fraser. Squeezing More from Limited Data with Recursive Transformers. *arXiv preprint arXiv:2608.26973*, 2026.

Alexander Hägele, Elie Bakouch, Atli Kosson, Loubna Ben Allal, Leandro von Werra, and Martin Jaggi. Scaling Laws and Compute-Optimal Training Beyond Fixed Training Durations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.18392.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training Large Language Models to Reason in a Continuous Latent Space. In *Conference on Language Modeling (COLM)*, 2025. arXiv:2412.06769.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training Compute-Optimal Large Language Models. *arXiv preprint arXiv:2203.15556*, 2022.

Benhao Huang, Chufan Shi, Junlin Chen, Shicheng Wen, Zhengzhong Liu, Eric Xing, and Xuezhe Ma. Towards Looped Models Done Right, Part II: Rethinking at Fixed Points. *arXiv preprint arXiv:2610.06833*, 2026.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*, 2020.

Harsh Kohli, Srinivasan Parthasarathy, Huan Sun, and Yuekun Yao. Loop, Think, & Generalize: Implicit Reasoning in Recurrent-Depth Transformers. In *Conference on Language Modeling (COLM)*, 2026. arXiv:2604.07822.

Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. *arXiv preprint arXiv:1909.11942*, 2019.

Ryan Lee, Jacob Biloki, Edward J. Hu, and Jonathan May. Sparse Layers are Critical to Scaling Looped Language Models. *arXiv preprint arXiv:2605.09165*, 2026.

Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Transformers Learn Shortcuts to Automata. *arXiv preprint arXiv:2210.10749*, 2022.

Joe Logan. Per-Token Fixed-Point Convergence in Depth-Recurrent Transformers. *arXiv preprint arXiv:2607.14427*, 2026.

Andrei Marchenko, Viacheslav Bezrukov, Oleg Kashurin, Inessa Fedorova, Dmitry Bocharov, Yuliana Shakhvalieva, Maria Tikhonova, and Valerii Ternovskii. Closing the Loop: Practical Training Recipes for Looped Language Models. *arXiv preprint arXiv:2610.00673*, 2026.

Sean McLeish, Arpit Bansal, Alex Stein, Neel Jain, John Kirchenbauer, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, Jonas Geiping, Avi Schwarzschild, and Tom Goldstein. Transformers Can Do Arithmetic with the Right Embeddings. *arXiv preprint arXiv:2405.17399*, 2024.

William Merrill and Ashish Sabharwal. The Expressive Power of Transformers with Chain of Thought. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.07923.

Giovanni Monea, Keshav Ramji, Yousef El-Kurdi, Luis A. Lastras, Yoav Artzi, Nathan Godey, and Ramón Fernandez Astudillo. The Surprising Effectiveness of Shared Memory in Looped Transformers. *arXiv preprint arXiv:2610.02383*, 2026.

nostalgebraist. interpreting GPT: the logit lens. LessWrong blog post, 2020. URL `https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens`.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show Your Work: Scratchpads for Intermediate Computation with Language Models. *arXiv preprint arXiv:2112.00114*, 2021.

Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. *arXiv preprint arXiv:2406.17557*, 2024.

Jacob Pfau, William Merrill, and Samuel R. Bowman. Let's Think Dot by Dot: Hidden Computation in Transformer Language Models. *arXiv preprint arXiv:2404.15758*, 2024.

Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt, and Yair Carmon. Resolving Discrepancies in Compute-Optimal Scaling of Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.19146.

Hayden Prairie, Zachary Novack, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. Parcae: Scaling Laws For Stable Looped Language Models. *arXiv preprint arXiv:2604.12946*, 2026.

Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with Latent Thoughts: On the Power of Looped Transformers. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2502.17416.

Avi Schwarzschild, Eitan Borgnia, Arjun Gupta, Furong Huang, Uzi Vishkin, Micah Goldblum, and Tom Goldstein. Can You Learn an Algorithm? Generalizing from Easy to Hard Problems with Recurrent Networks. *arXiv preprint arXiv:2106.04537*, 2021.

Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. How Much Is One Recurrence Worth? Iso-Depth Scaling Laws for Looped Language Models. *arXiv preprint arXiv:2604.21106*, 2026.

Shaowen Wang, Bingrui Li, Ge Zhang, Wenhao Huang, Shen Yan, and Jian Li. On the Residual Scaling of Looped Transformers: Stability and Transferability. *arXiv preprint arXiv:2606.18524*, 2026a.

Shaowen Wang, Ge Zhang, Kairong Luo, Yuhao Wu, Shaofan Liu, Jiaheng Liu, Wenhao Huang, Shen Yan, and Jian Li. SMELT: Scaling Laws for Compute-Matched MoE Looped Transformers. *arXiv preprint arXiv:2609.01343*, 2026b.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv preprint arXiv:2201.11903*, 2022.

Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. Parallel Loop Transformer for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2510.24824*, 2025.

Qifan Yu, Zhenyu He, Sijie Li, Xun Zhou, Jun Zhang, Jingjing Xu, and Di He. Enhancing Auto-regressive Chain-of-Thought through Loop-Aligned Reasoning. *arXiv preprint arXiv:2502.08482*, 2025.

Tong Zhang, Junhao Hu, Yun Peng, and Tao Xie. When Does Recurrence Become an Algorithm? Convergence Selection in Weight-Tied Looped Transformers. *arXiv preprint arXiv:2607.20594*, 2026.

Yongchao Zhou, Uri Alon, Xinyun Chen, Xuezhi Wang, Rishabh Agarwal, and Denny Zhou. Transformers Can Achieve Length Generalization But Not Robustly. *arXiv preprint arXiv:2402.09371*, 2024.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling Latent Reasoning via Looped Language Models. *arXiv preprint arXiv:2510.25741*, 2025.

A The pre-registered hypotheses and decision rules

The pre-registration (frozen 2026-09-23; an English translation of the binding original is in the repository with the commands that verify its hash) fixed the grid of Section 3.2, the fitting protocol of Section 3.3 and these decision rules.

Hypothesis	Decision rule for support
H1: truncated backpropagation lowers φ	$\varphi_{\text{trunc}} < \varphi_{\text{full}}$ with non-overlapping 95% bootstrap intervals, under both equal-token and equal-FLOP accounting; if only under equal tokens, the effect is attributed to the reallocation of FLOPs.
H2: a looped middle block has a higher φ than a looped whole stack	$\varphi_{\text{mid}} > \varphi_{\text{whole}}$ with non-overlapping intervals; raw-loss pairs of similar executed depth (middle block $r = 8$ vs whole stack $r = 4$; middle block $r = 4$ vs whole stack $r = 2$) are reported alongside, and a disagreement is reported as "depends on the measure".
H3: φ saturates in r	The geometric form has the lower extrapolation error in both directions (10M–40M predicting 80M; 20M–80M predicting 10M) and an information-criterion difference above 10.
H4: the compute-optimal r	Under matched training FLOPs the optimal $r \leq 2$, under matched deployment FLOPs larger; computed from the fitted law, without a test.

A factor effect counts only if it exceeds both the bootstrap half-width and the spread of φ fitted separately to the three seeds at 20M and 40M. The information criterion of H3 turned out to be computable only as a proxy from the Huber loss, which is not a likelihood; it is not used as evidence (Appendix G). Two rules were added during the grid, after the first spikes (2026-09-26) and after the collapse (2026-09-30), each before the results it governs existed, and are dated in the deviation log: spikes are reported per run with a robustness fit without the runs that have them, and a result whose cooldown started after a collapse enters no fit (one run, a 40M whole stack looped 8 times with truncation, seed 43).

B Training details

Learning rates. A sweep of 33 runs at $10N$ (30 dense runs at five rates per width, a $\beta_2 = 0.99$ group at 20M, and three looped runs at 80M) fixed the rate per width (Table 1); the minima of all five widths lie

inside the swept range, and $\beta_2 = 0.95$ beat 0.99 by 0.008 nats. The hidden layers use $\eta_0(8/12)^{-1/2} \approx 1.22\,\eta_0$. A check for looped cells, whose decision rule was written before its runs on 2026-09-25, two days after the pre-registration (20M, ten cells at 0.5, 0.71 and 1 times the dense rate, $10N$) found no better rate by the rule fixed in advance (pooled best multiple 0.97, threshold 0.84), so all looped language models use the dense rates.

FLOP accounting. Training FLOPs count two FLOPs per multiply-add over every executed layer, the attention terms and the output head, plus a backward pass of twice the forward cost for every layer that receives gradients; with truncated backpropagation the backward pass covers the prelude, the coda, the output head and the last $r/2$ loops.

The grid at equal parameters. Table 7 gives the looped minus dense loss per cell and size, averaged over the three budgets.

Table 7: Looped minus dense loss at equal parameters and tokens, mean over the budgets $10N$, $20N$ and $40N$ and over seeds (negative: looped better). *Includes a seed with a severe loss spike; without it, −0.029.

Cell	10M	20M	40M	80M
Middle block, $r = 2$	+0.068	+0.025	−0.010	−0.015
Middle block, $r = 4$	+0.055	+0.002	−0.030	−0.039
Middle block, $r = 4$, truncated	+0.079	+0.031	+0.006	+0.009
Middle block, $r = 8$	+0.031	−0.003	−0.033	−0.033
Middle block, $r = 8$, truncated	+0.039	+0.001	−0.022	−0.018
Whole stack, $r = 2$	+0.172	+0.079	+0.021	+0.003
Whole stack, $r = 4$	+0.150	+0.085	+0.047	+0.025
Whole stack, $r = 4$, truncated	+0.044	+0.003	−0.014	+0.017
Whole stack, $r = 8$	−0.074	−0.055	+0.213*	−0.003
Whole stack, $r = 8$, truncated	−0.053	−0.041	−0.042	−0.020

Loss spikes. 30 of the 111 runs had at least one loss spike: 25 with truncated backpropagation (16 of these whole-stack loops) and five whole-stack loops with $r = 8$ and full backpropagation; one run collapsed. In an exploratory follow-up, a 10M whole stack looped 4 times with truncation, trained at half the learning rate in three seeds, had no spike at all (4, 1 and 10 at the original rate); it was worse by 0.044 nats at $10N$ and better by 0.064 at $40N$, which cannot be credited to the missing spikes alone, since longer budgets also favour a lower constant rate.

C Fits and robustness

The primary ruler, fitted to 54 dense results, is $E = 0.050$ (at the lower bound of the fit), $A = 6.98$, $\alpha = 0.054$, $B = 1065$, $\beta = 0.370$; the joint fit over all 352 results gives $E = 1.94$, $A = 61.7$, $\alpha = 0.267$, $B = 197$, $\beta = 0.266$. Under the constant-φ model, setting every φ to 0 raises the Huber loss only from 0.00352 to 0.00379, and setting it to 1 raises it to 0.0142. Table 8 gives the primary φ under the pre-registered robustness variants. Fitted separately to each seed, φ varies by 0.10–0.21 per cell at 20M and 0.00–0.11 at 40M; the factor effects of Table 2 (0.13–0.17) do not exceed the 20M spread.

For H3, the pre-registered extrapolation (10M–40M to 80M and 20M–80M to 10M, with the dense-only 160M size in neither training set) gives a root-mean-square error of 0.0096 and 0.0200 for r^{φ} and 0.0085 and 0.0203 for the geometric form. Five of the seven robustness variants show the same split; without $r = 8$ the split is reversed, and on the second validation set r^{φ} is better in both directions. For H4, the fitted law makes the dense model optimal at every training budget from 10^{17} to 10^{20} FLOPs, with the smallest available r the best looped choice, 0.00–0.17 nats behind; under deployment compute the dense model is optimal as well, with either set of ruler parameters.

Table 8: φ (primary fit) under robustness variants, in the order middle block full / middle block truncated / whole stack full / whole stack truncated.

Variant	φ
Primary analysis	0.13 / 0.00 / 0.00 / 0.17
Without the 30 runs with loss spikes	0.13 / 0.11 / 0.00 / 0.22
Mean of the last three evaluations	0.14 / 0.00 / 0.00 / 0.17
Second validation set (SlimPajama)	0.31 / 0.18 / 0.02 / 0.30
Without $r = 8$	0.07 / 0.00 / 0.00 / 0.07
Half the budget	0.08 / 0.00 / 0.00 / 0.15
N without the output head	0.07 / 0.00 / 0.00 / 0.14
Equal-FLOP accounting	0.13 / 0.08 / 0.00 / 0.16

D Learning-rate re-tuning (S1)

Table 9 gives the gap between the middle block looped 4 times and the dense model at the shared rate (Δ_1) and with each model at its best rate ($\Delta^\star$), and $h = \Delta_1 - \Delta^\star$, the bias of the shared rate against the looped model. The pre-specified readings: the trend with size holds (at $40N$ the improvement from 10M to 40M is 0.050 against 0.065 at the shared rate, above the threshold of half of it), the trend with data holds and grows (10M from $10N$ to $160N$: 0.141 against 0.119), the equal-compute verdict is unchanged (its 10M comparison at $160N$, extrapolated then, was later measured as parity; Section 4.3), and the bias exceeds the 0.01 tolerance at 10M only.

Table 9: S1: looped (middle block, $r = 4$) minus dense loss at the shared rate and at each model's best rate (seed 42; 20M: mean of seeds 42 and 43). †The lowest rate tested.

Size, budget	Δ_1, shared rate	$\Delta^\star$, best rates	h	Best rate, dense / looped
10M, $10N$	+0.082	+0.082	+0.000	1 / 0.71
10M, $20N$	+0.044	+0.026	+0.018	1 / 0.5
10M, $40N$	+0.015	−0.005	+0.020	1 / 0.5
10M, $80N$	−0.010	−0.035	+0.024	1 / 0.35†
10M, $160N$	−0.037	−0.059	+0.022	1 / 0.35†
20M, $10N$	+0.028	+0.028	0.000	1 / 1
20M, $20N$	−0.005	−0.005	0.000	1 / 1
20M, $40N$	−0.031	−0.031	0.000	1 / 1
40M, $10N$	−0.008	−0.008	0.000	1 / 1
40M, $20N$	−0.032	−0.036	+0.004	1 / 0.71
40M, $40N$	−0.050	−0.055	+0.005	0.71 / 0.71

On the trunk checkpoints at $32N$, the range of the main-set loss over 4–16 inference loops is 0.30, 0.36, 1.02, 3.39 and 4.10 nats at 0.35, 0.5, 0.71, 1 and 1.41 times the dense rate at 10M, 0.53–3.42 at 20M and 0.77–4.81 at 40M (0.5 to 1.41 times): lower rates make the iteration closer to convergent, but at every rate the better of 8 and 16 loops is worse than the 4 loops of training, by 0.07–3.0 nats.

E The synthetic task

Data and training. Tokens are the values 0–96 and a few markers (start, separator, lookup, answer and padding); a problem is a start value followed by d steps of the form `LUT k ;`, with the table index k written as one of the value tokens 0–31, and the answer marker. Problems are drawn online. During the first half of training the largest step count grows linearly from 1 to 8; the second half draws uniformly from 1–8. Models have width 256 and attention heads of dimension 64, and the looped models use the input injection and residual scaling of the language models. Training uses AdamW ($\beta = (0.9, 0.95)$, weight decay 0.1, gradient clipping 1.0), 400 warm-up steps and a cosine decay to a tenth of the peak rate, 512 problems per step, 32,000 steps. The two deepest conditions of S5 and S9 accumulated gradients over two half-batches to fit

in memory (the same loss and update). Probes are linear maps from the state after each loop (or layer) to the 97 values, trained on 4000 problems with 8 steps and tested on 1000, at the table-index, separator and next-operation positions of each step, keeping the best.

Learning rates. The sweep (seed 1; 2.5×10^{-4}, 5×10^{-4} and 10^{-3}, extended at an edge) chose by mean in-range accuracy at the condition's own loop count: dense 8 and 20 layers 10^{-3}, dense 36 layers 5×10^{-4}, looped $r = 2$ and $r = 8$ 2.5×10^{-4}, looped $r = 4$, random loop count and loops = steps 1.25×10^{-4}; in S5 the chain-of-thought models 1.25×10^{-4} and the pause model 5×10^{-4}; in S9 1.25×10^{-4}. The criterion saturates at low rates (several conditions score about 0.999 at two or three rates), and ties went to the lowest rate by implementation, a rule the protocol had not specified. Extrapolation is sensitive to this choice: in the sweep, the accuracy of the loops = steps model on 9 to 14 steps fell from 1.00 to 0.91 at 2.5×10^{-4} but from 1.00 to 0.05 at 1.25×10^{-4}.

The pre-specified predictions. Table 10 lists the verdicts.

Table 10: The predictions of the frozen synthetic design and their verdicts.

Prediction	Verdict
P1: the largest solvable length rises from dense 8 layers to $r = 2, 4, 8$; $r = 8$ reaches 16	Holds in direction (4.2, 6.4, 8.0, 9.0); $r = 8$ does not reach 16.
P2: dense models of equal depth are not worse, and the gap is at most a quarter of the dense-8 to looped gap	Not supported as stated: they solve 1–1.6 fewer steps, within the tolerance for $r = 8$, not for $r = 4$.
P3a: fixed-loop models lose accuracy sharply at other loop counts	Fails in the direction that favours looping: with 16 loops the 8-loop model keeps 0.997 in range and solves 11.0 steps instead of 9.
P3b: the random-loop model does not improve with more loops beyond 8 steps	Fails: it gains on long chains (0.08 to 0.49 at 16 steps).
P3c: the loops = steps model with d loops beats 8 loops beyond 8 steps, to at least 16	Holds (by 0.34–0.95 at every length from 10 to 16).
P4: only the loops = steps model's probe table is diagonal	Half right: diagonal shares 0.95 (loops = steps) and 0.97 (8 loops), 0.12 for the others.
P5: the gaps are much larger than on text	Holds.

Supplementary predictions. S2: a high learning rate mainly prevents learning (Section 6.4). S5 held one of three: the 8-layer chain-of-thought model was predicted to extend at least as far as the looped models (≥ 0.7 at 12 steps) and reached 0.014; pause tokens were predicted to add less than 0.1 and added nothing; the 36-layer chain-of-thought model was predicted to reach 0.8 at 12 steps and reached 0.017. S8 held one of five (Table 11). S9 held three of four; the one that failed is the frozen state rule's stop (0.52 at 12 steps against a predicted 0.6).

Table 11: S8, all rule families on the reporting set: accuracy at 12 / 16 steps (mean loops used at 12 steps). The rule the plan's selection criterion picks is in the last column.

Model	Own loops	Best fixed	State < 0.02	Prediction unchanged	Confidence ≥ 0.99	Selected
$r = 2$	0.05 / 0.04	0.62 / 0.33	0.45 / 0.20 (29)	0.56 / 0.21 (6.3)	0.70 / 0.30 (8.3)	confidence ≥ 0.9
$r = 4$	0.04 / 0.02	0.83 / 0.29	0.65 / 0.21 (11)	0.48 / 0.09 (6.8)	0.81 / 0.26 (9.2)	vote, loops 8–32
$r = 8$	0.04 / 0.03	0.86 / 0.37	0.71 / 0.23 (18)	0.09 / 0.04 (5.5)	0.84 / 0.37 (17)	vote, loops 8–32
Random	0.77 / 0.08	0.83 / 0.48	0.73 / 0.37 (11)	0.28 / 0.11 (5.0)	0.83 / 0.47 (14)	vote, loops 8–32
Loops = steps	0.96 / 0.36	0.96 / 0.36	0.03 / 0.03 (25)	0.21 / 0.06 (8.8)	0.76 / 0.28 (14)	none eligible

Pointer chasing. The design's second task, following a random function given in the prompt for k hops, was not learned by any condition under the recipe calibrated on the chain task (0.10–0.24 accuracy, 0.10–0.11 on a single hop), and a check showed that even single-hop retrieval was not learned in that format. It was dropped from the study before any comparison on it.

F Deployment measurements

Inference was timed on one RTX 4090 (all other GPUs idle) in bf16, with the decoding step in a CUDA graph, one KV-cache slot per executed layer, random weights, a 512-token prompt and 512 generated tokens, at batch 1 and 32. Table 12 gives the ratios at equal loss for all middle-block and whole-stack cells, against the dense model that reaches the same loss at $40N$ (interpolated between dense sizes).

Table 12: S3: looped over dense, at equal loss. *160M lies beyond the largest dense model (extrapolated).

		Weight	KV cache	Decoding time	Peak memory	
Cell	Sizes	memory	per token	batch 1	batch 1	batch 32
Middle block, $r = 2$	20M–80M	0.93–1.03	1.43–1.51	1.45–1.49	0.99–1.15	1.21–1.24
Middle block, $r = 4$	20M–160M	0.81*–0.96	2.20–2.41	2.37–2.47	0.98*–1.20	1.57–1.58
Middle block, $r = 8$	20M–80M	0.83–0.92	4.01–4.23	4.27–4.56	1.08–1.34	2.29–2.40
Whole stack, $r = 4$	20M–80M	1.03–1.11	4.02–4.27	4.00–4.11	1.18–1.84	2.32–2.41
Whole stack, $r = 8$	20M–80M	0.81–0.85	6.96–7.23	7.57–7.91	1.27–1.50	3.38–3.91

For KV sharing (S6), the first 262,144 tokens of the main validation set were decoded token by token in 256 windows of 1024, with one cache slot per executed layer (whose loss equals that of the ordinary forward pass to four decimals) and with one slot per unique layer, written at a position's last loop and read by every later loop. The models are the 20M trunk checkpoints at $32N$ (seed 42); Figure 8b gives the loss differences, and the random-loop model run with 16 loops instead of 8 also shares at no cost.

G Deviations and corrections

Every item is dated in the deviation log in the repository.

- Two power failures of the training machine cut CPU speed for a time; runs resumed from checkpoints and no result was affected.
- The synthetic task changed twice before its formal runs, both times for learnability and before any comparison between conditions: a uniform mixture of 1–8-step problems was not learned (a curriculum was added), and modular arithmetic chains were learned too slowly (fixed random lookup tables replaced them). Training was lengthened to 32,000 steps for every condition, three seeds were added to the first two (declared before their results), and the probe was widened from one to three positions per step after a diagnostic showed values stored at other positions.
- The learning-rate criterion of the synthetic task saturated, and ties went to the lowest rate by implementation (Appendix E).
- A first review (2026-10-14) found five errors: the verdict on the random-loop model's gain from extra loops used the longest solvable chain instead of the accuracy difference the design specified (by the latter it gains); the batch-1 peak memory of S3 counted two KV caches (re-measured); the information criterion of H3 is a Huber-loss proxy (no longer cited); the extrapolation test of H3 kept the 160M size in its training set (redone as pre-registered); and the design said problems never repeat, which is false for short ones.
- A second review (2026-10-15) corrected about 35 numbers and statements, among them: P3c holds by its own decision quantity; P4 is judged by the design's diagonal share; the loops = steps model keeps its values within its d loops (an earlier text said it overwrites them); the 12-step comparison with dense models had mixed in extra loops; the best over loop counts had left out 4 and 8 loops for $r = 2$; and the equal-compute claim was limited to the measured range.
- An evaluation the frozen synthetic design specified, the state-rule stop of the loops = steps model, was computed but not reported until the second review.

- Two synthetic conditions of S5 and S9 ran out of GPU memory at the planned batch, having been tested only on a CPU before their plan was frozen; they were rerun with two accumulated half-batches.
- One unexplained anomaly: at 160M one looped seed is 0.2–0.6 nats worse than the other on the out-of-distribution sets at $40N$, while their main-set losses are identical.